COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	09/06/2026	**DATE SUBMITTED:**	09/07/2026
NAME:	Almirol Joel A	**INSTRUCTOR:**	Engr.JIMLORD QUEJADO

ACTIVITY NO. 7.1 Sorting Algorithms pt1

1. PROCEDURE OUTPUT

What procedures did you perform?

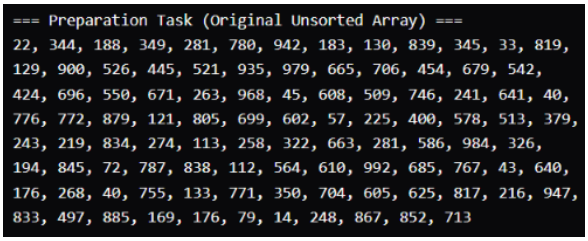
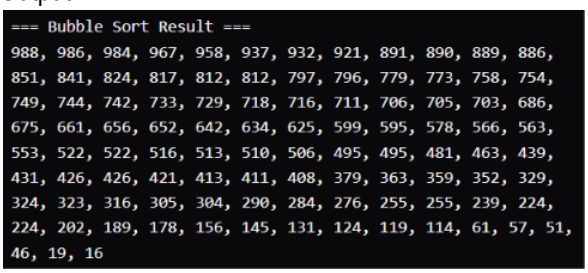
Code + Console Screenshot	 Output: 
Observations	The program creates a baseline random array of integers between 0 and 999, with size 100, using std::Srand() % 1000 seed initialized with the system clock time. The generated data is fully randomized and not structured with any initial ordering (22, 344, 188, ...)

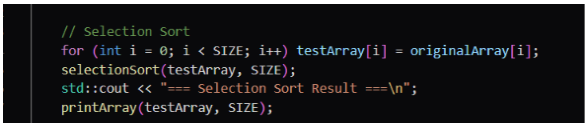
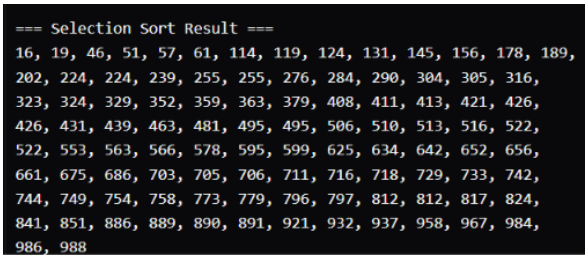
Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	 Output: 
Observations	The Bubble Sort algorithm sorts the array in descending order, beginning from the largest number (988) to the smallest one (16). This is due to the swapping condition arr[j] > arr[i], which moves the larger numbers toward the left-hand side of the array on every outer loop iteration.

```
// Bubble Sort Technique
for (int i = 0; i < SIZE; i++) testArray[i] = originalArray[i];
bubbleSort(testArray, SIZE);
std::cout << "=== Bubble Sort Result ===\n";
printArray(testArray, SIZE);
```

```
=== Bubble Sort Result ===
988, 986, 984, 967, 958, 937, 932, 921, 891, 890, 889, 886,
851, 841, 824, 817, 812, 812, 797, 796, 779, 773, 758, 754,
749, 744, 742, 733, 729, 718, 716, 711, 706, 705, 703, 686,
675, 661, 656, 652, 642, 634, 625, 599, 595, 578, 566, 563,
553, 522, 522, 516, 513, 510, 506, 495, 495, 481, 463, 439,
431, 426, 426, 421, 413, 411, 408, 379, 363, 359, 352, 329,
324, 323, 316, 305, 304, 290, 284, 276, 255, 255, 239, 224,
224, 202, 189, 178, 156, 145, 131, 124, 119, 114, 61, 57, 51,
46, 19, 16
```

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	 Output 
Observations	Selection Sort algorithm sorts the elements in an increasing manner, starting from 16 to 988. This is done using the Routine Smallest() function that helps in locating the smallest number each time from the unsorted part and placing it in its appropriate position.

```
// Selection Sort
for (int i = 0; i < SIZE; i++) testArray[i] = originalArray[i];
selectionSort(testArray, SIZE);
std::cout << "=== Selection Sort Result ===\n";
printArray(testArray, SIZE);
```

```
=== Selection Sort Result ===
16, 19, 46, 51, 57, 61, 114, 119, 124, 131, 145, 156, 178, 189,
202, 224, 224, 239, 255, 255, 276, 284, 290, 304, 305, 316,
323, 324, 329, 352, 359, 363, 379, 408, 411, 413, 421, 426,
426, 431, 439, 463, 481, 495, 495, 506, 510, 513, 516, 522,
522, 553, 563, 566, 578, 595, 599, 625, 634, 642, 652, 656,
661, 675, 686, 703, 705, 706, 711, 716, 718, 729, 733, 742,
744, 749, 754, 758, 773, 779, 796, 797, 812, 812, 817, 824,
841, 851, 886, 889, 890, 891, 921, 932, 937, 958, 967, 984,
986, 988
```

Table 7-3. Selection Sort Algorithm

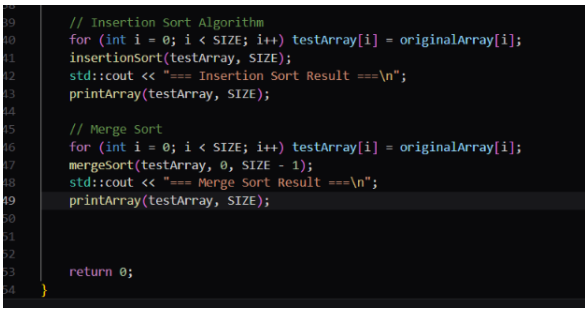
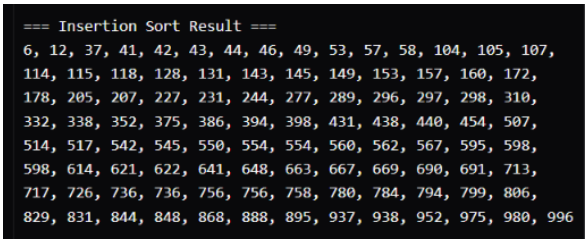
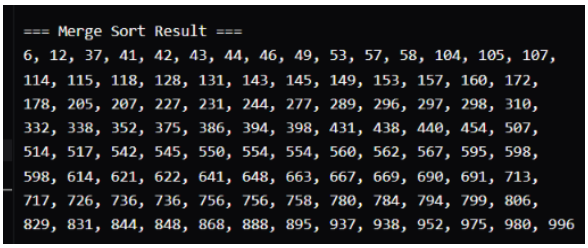
Code + Console Screenshot	 Output:  
Observations	Both Insertion Sort and Merge Sort generate similar output arrays that are well sorted in an ascending manner, beginning from 6 and ending with 996. The difference is that Insertion Sort does it through swapping neighboring elements backwards, while Merge Sort employs recursion and divide-and-conquer strategy.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Pseudocode:

```
ALGORITHM CountVotesAndDetermineWinner(A, N)
   // Sort the vote array in ascending order using
Insertion Sort
   FOR K = 1 TO N - 1 DO
      temp = A[K]
      J = K - 1
      WHILE J >= 0 AND temp <= A[J] DO
         A[J + 1] = A[J]
         J = J - 1
      END WHILE
      A[J + 1] = temp
      K = K + 1
   END WHILE

   //  Initialize tally counts for Candidate IDs (1 to 5)
   FOR c = 1 TO 5 DO
      VOTE_COUNT[c] = 0
   END FOR

   //  Iterate through the sorted array to count votes
   FOR i = 0 TO N - 1 DO
      candidateID = A[i]
      VOTE_COUNT[candidateID] =
VOTE_COUNT[candidateID] + 1
   END FOR

   // Search for the candidate with the highest vote
count
   maxVotes = -1
   winningCandidate = 1

   FOR c = 1 TO 5 DO
      IF VOTE_COUNT[c] > maxVotes THEN
         maxVotes = VOTE_COUNT[c]
         winningCandidate = c
      END IF
   END FOR

   // Output results and announce the winning
candidate
   PRINT "Sorted Votes:", A
   PRINT "Tally Results:", VOTE_COUNT
   PRINT "Winner:", winningCandidate, "with",
maxVotes, "votes"
```

```cpp
#include <iostream>
#include <string>
#include <ctime>
#include "SortAlgorithms.h"

int main() {

    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    // VOTE COUNTING SYSTEM
    const int VOTE_SIZE = 101;
    int votes[VOTE_SIZE];

    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    // Generate 101 random votes ranging from 1 to 5
    for (int i = 0; i < VOTE_SIZE; i++) {
        votes[i] = (std::rand() % 5) + 1;
    }

    // Sort the vote array using Insertion Sort
    insertionSort(votes, VOTE_SIZE);

    // Count frequencies per candidate
    int voteCounts[6] = {0};
    for (int i = 0; i < VOTE_SIZE; i++) {
```

```cpp
    for (int i = 0; i < VOTE_SIZE; i++) {
        voteCounts[votes[i]]++;
    }

    // Determine winner
    int maxVotes = -1;
    int winningCandidateID = 1;

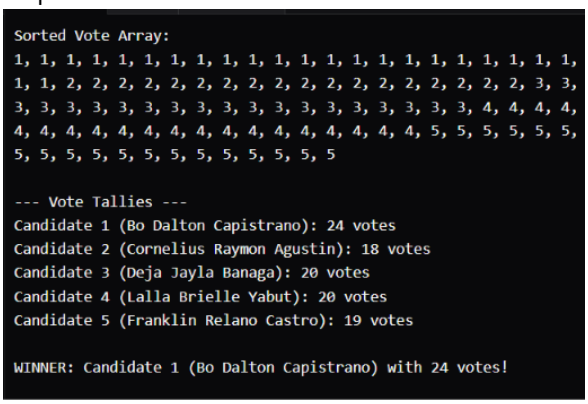
    for (int c = 1; c <= 5; c++) {
        if (voteCounts[c] > maxVotes) {
            maxVotes = voteCounts[c];
            winningCandidateID = c;
        }
    }


    std::cout << "Sorted Vote Array:\n";
    for (int i = 0; i < VOTE_SIZE; i++) {
        std::cout << votes[i] << (i == VOTE_SIZE - 1 ? "" : ", ");
    }
    std::cout << "\n\n--- Vote Tallies ---\n";

    for (int c = 1; c <= 5; c++) {
        std::cout << "Candidate " << c << " (" << candidates[c - 1] << "): "
                  << voteCounts[c] << " votes\n";
    }

    std::cout << "\nWINNER: Candidate " << winningCandidateID << " ("
              << candidates[winningCandidateID - 1] << ") with "
              << maxVotes << " votes!\n";

    return 0;
```

END ALGORITHM	
	Output 

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	Candidate 1 : 24	Candidate 1 : 24
	Candidate 2:18	Candidate 2 : 18
	Candidate 3:20	Candidate 3 : 20
	Candidate 4:20	Candidate 4 : 20
	Candidate 5:19	Candidate 5 : 19

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, the developed voting algorithm was indeed very effective. By sorting the unsorted array before counting, it is possible to organize the randomly created votes into organized blocks such that each block will consist of similar numbers, all 1's first, then all 2's, and so forth. As such, the algorithm is able to iterate through the array in one pass without performing any messy searches through unorganized and unsorted arrays. Using the array frequency to count candidate votes makes finding the candidate with maximum votes

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.